研序 ScholarFlow

面向科研阅读与项目执行的 Windows 桌面 Agent · 技术文档

版本：2026-10-31 Windows 桌面版

作者：张祎凡（个人开发）

本文用于 AI 大模型创新应用挑战赛科研助手 Agent
赛道的技术材料提交，覆盖项目概述、需求分析、系统架构、AI
技术方案、核心实现、测试部署、创新点和后续计划。

1. 项目概述

研序 ScholarFlow 是面向科研人员的桌面端 Agent。它将 PDF 文献导入、细粒度解析、关键词与向量混合检索、抗幻觉引用问答、原文版式定位、引用图谱和项目协作串成一条工作流。产品目标是让用户从“找到论文”进一步走到“用论文证据完成任务”，并在模型输出、文件修改和 Git 操作之间保留可审阅、可恢复的中间状态。

当前交付范围为 Windows 桌面端。应用由 FastAPI 后端、Vue 3 前端和 pywebview 启动器组成，打包后形成 ScholarFlow.exe。无 API Key 时进入离线 Mock 模式，可完成基本演示；配置 OpenAI 兼容端点后可切换真实模型。

核心价值

1. 证据可追溯：回答中的引用可以跳转到论文页码和版式坐标。
2. 工作流可恢复：任务、草稿、工具结果和 Git 操作均有持久化状态。
3. 零 Key 可演示：离线模式保留检索、取证、引用和拒答链路。
4. 桌面端一体化：文献、项目文件、终端和 Git 在同一窗口内协作。

2. 需求分析

科研阅读通常需要在 PDF 阅读器、搜索工具、笔记软件和代码工作区之间来回切换。普通聊天模型能够生成流畅文本，但难以保证每个论断都能回到原文；长对话还会遇到上下文膨胀、失败重试和任务结果丢失问题。桌面开发场景又要求文件改动、Git 写入和命令执行先审阅、可恢复，且不应把密钥放进源码。

功能需求

- 导入 PDF，提取页码、段落、参考文献和版式坐标。
- 使用 BM25/关键词与本地或 API 向量进行混合检索，并保留证据来源。
- Agent 根据证据组织答案，逐句附带引用；证据不足时拒答。
- 点击引用回到原文页码和高亮框，支持引用图谱。
- 支持项目文件浏览、只读搜索、文件变更审阅、Git 状态/差异/提交和工作树。
- 支持执行/计划模式、审批、SSE 流式输出、前台和后台检查点恢复。
- Windows EXE 可离线运行，数据和配置保存在用户数据目录。

非功能需求

交互需要适配 1440×900 和 1100×700 桌面窗口；长任务不能因单次请求失败而丢失；同一数据目录只允许一个后端实例；敏感配置不得进入公开仓库；源码应能从 README 重建开发环境。

3. 系统架构设计

层	主要技术	责任
桌面层	pywebview、Windows EXE、PowerShell/cmd 辅助进程	创建原生窗口、托管静态前端、承载终端
表现层	Vue 3、Vue Router、Element Plus、Vite	文献库、阅读器、聊天、项目、Git、任务和设置
API 层	FastAPI、SSE、Pydantic	会话、文献、项目、审批、任务和状态接口
Agent 层	Planner、Executor、Critic、检查点	规划、工具调用、证据核验、恢复和重试
检索层	PDFPlumber、切块、BM25、向量、RRF	文本解析、排序、证据对象和引用闸门
数据层	SQLite、NumPy、本地文件	论文、片段、问答、任务、草稿、配置和向量
外部能力	内置 MCP、可选自定义 MCP、OpenAI 兼容接口	学术搜索、网页抓取和真实模型调用

数据流

PDF 进入解析器后被切成带文献 ID、页码和坐标的片段；关键词索引与向量索引分别计算候选，再使用 RRF 合并。Agent 将候选证据交给模型或 Mock
模型生成答案，引用闸门检查每个事实性句子是否带有证据标记。前端收到 SSE
增量后更新思考、工具和答案区域；所有关键状态同时写入 SQLite 或本地配置，供重启恢复。

桌面实例与恢复

启动器为数据目录创建实例锁和运行时记录。同一目录重复启动时请求已有窗口，不创建第二个后端；不同 DATA_DIR 可以并行运行。任务在工具调用、钩子和回答提交阶段保存检查点，重启后显示“核对后继续”“跳过”或“从头运行”，不会盲目重复外部写操作。

4. AI 技术方案

4.1 模型选型理由

采用 OpenAI 兼容 Chat Completions 协议，是因为它同时覆盖云端供应商和本地 vLLM，切换模型只需替换 Base URL、模型名和 Key。应用默认提供 Mock LLM，保证评审在无网络、无 Key
条件下仍能演示完整链路；真实模型则用于自然语言归纳、跨论文比较和工具选择。Embedding 同样支持 API 与本地确定性哈希向量两种模式，降低首次运行成本。

4.2 集成方式

LLM 抽象层统一 chat 和 stream_chat，解析普通回答、推理内容、工具调用和 usage。Agent
工具通过函数描述注册，模型选择检索、项目读取、MCP 或文件操作。内置 MCP 在进程内提供 arXiv、Semantic Scholar、Crossref 和网页抓取工具；外部 MCP 可在设置页按 stdio 或 streamable-http 接入。

4.3 抗幻觉与证据链

检索候选包含文献、页码、段落、引用关系和版式坐标。RRF
融合关键词分数与向量分数，避免单一检索方式漏召回。Agent 先取证再作答；引用闸门按句检查引用标记和证据映射，无证据时走拒答路径。用户点击引用时，前端使用证据坐标定位到 PDF
原文，形成“答案—证据—页码—原文”的闭环。

4.4 优化策略

- 通过本地 Mock 和本地向量实现零 Key 冷启动，减少评审环境不确定性。
- 通过流式输出降低首字延迟，同时把思考、工具和答案分开渲染。
- 长会话采用增量摘要，默认保留最近四轮，原始对话和证据不删除，可按需回查。
- 每个任务保留检查点、操作 ID 和版本号，利用幂等请求和冲突检测避免重复执行。
- 对文件、Git、终端和 MCP 写操作使用审批策略；计划模式只开放读取能力。

5. 核心功能实现

5.1 PDF 解析与版式阅读

backend/app/parsing/
负责文本、页码和结构化内容提取；解析结果写入本地文献目录和数据库。前端阅读器使用 pdf.js
渲染页面，根据证据对象的页码与 bbox 绘制高亮。参考文献和库内论文关系被转换为图谱节点和边。

5.2 混合检索与问答

切块阶段保留上下文窗口和来源元数据。关键词检索适合精确术语，向量检索适合语义相近表达；二者通过 RRF 合并。Agent 循环由 Planner、Executor 和 Critic 组成，支持工具调用、重试、证据去重和拒答。离线 Mock
模型会真实发起 retrieve_evidence，再把检索原文拼接成带引用答案。

5.3 项目协作与 Git

项目层提供目录浏览、UTF-8 文本预览、ripgrep 搜索和最多 8
个引用文件。文件变更先产生差异提案，确认后才写入；Git
页面支持状态、差异、暂存、取消暂存和提交。工作树支持分离 HEAD 或新分支，迁移功能在相同提交或完整分支/提交关系下搬运暂存、未暂存和未跟踪字节，并保存备份与恢复日志。

5.4 任务恢复和终端

前台回答和后台任务都使用 SQLite 持久化状态。SSE 断线后客户端按版本号重连，旧响应不能覆盖新草稿。终端由独立辅助进程承载，限制输出缓存和会话数量；Windows 上使用系统 PowerShell 或
cmd，命令执行仍遵循当前用户权限。

6. 测试与部署

6.1 测试结果

本版本后端全量单元测试记录为 494 passed。Workspace Transfer 专项记录为 29 passed。前端生产构建成功，TypeScript 检查、Markdown 渲染、项目/Git、文件变更、计划模式、归档、搜索、任务恢复和桌面状态专项均有独立日志。Windows EXE 冒烟验证了旧数据库迁移、换端口冷启动、实例归属、计划与摘要恢复、前后台任务恢复、文件变更/Git 状态以及迁移重试和回退；静态资源与当前前端构建逐字节一致。

测试使用隔离 SQLite、确定性模型夹具和临时项目目录，不把离线夹具结果描述为真实供应商模型质量，也不把浏览器自动化截图描述为原生窗口逐像素验收。当前桌面范围明确为 Windows，不包含移动端。

6.2 源码运行

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest
uvicorn app.main:app --host 127.0.0.1 --port 8765

cd ..\frontend
npm install
npm run dev
```

6.3 配置与安全

根目录 .env.example 给出 DATA_DIR、LLM_API_KEY、LLM_BASE_URL、LLM_MODEL、EMBEDDING_API_KEY、EMBEDDING_BASE_URL、EMBEDDING_MODEL、REDIS_URL、Agent 步数和检索数量。桌面 EXE 默认把数据放在 %APPDATA%\ScholarFlow。Key 会脱敏展示，只通过本地环境变量或应用设置保存，不进入源码和公开仓库。

7. 技术创新点

- 证据坐标闭环：把检索片段、引用编号、页码和 PDF 版式坐标统一成可回溯对象，答案可以直接回到原文。
- 离线可复现 Agent：无 Key 时仍保留“检索—取证—作答—引用校验”的真实链路，而不是只展示静态演示文本。
- 可恢复的工具执行：把工具调用、文件变更、Git 写入和任务状态做成带版本号的检查点，支持核对、继续、跳过和回退。
- 桌面工作区安全边界：项目、终端、MCP 和 Git 操作统一进入审批策略；计划模式从工具层限制写入。

8. 应用创新点

- 将科研阅读与项目开发放在一个工作区，用户可以从论文证据直接进入项目文件和实验任务。
- 通过引用图谱、原文定位和跨论文检索降低整理综述的切换成本。
- 同时覆盖个人离线演示和真实模型接入，适合竞赛评审、课堂教学和无稳定外网的科研环境。
- 以 Windows 原生桌面体验承载长会话、终端、Git 和工作树，面向实际开发流程而非单纯聊天窗口。

9. 当前限制与部署计划

当前提交未包含公网在线演示地址、评审账号和正式 1080p
MP4；这些需要部署平台和录制条件，不能用本地地址替代。尚未取得完整 Codex
原生界面基准截图，因此不宣称像素级一致。模型回答质量、外部 MCP 稳定性和超大 PDF 的解析耗时仍需要真实数据持续评估。后续计划包括完成公网部署与监控、录制正式演示视频、增加更多解析器和检索评测集，并继续根据可验证的桌面对标证据调整交互。

10. 总结

研序 ScholarFlow 已形成 Windows 桌面端的科研 Agent
闭环：文献进入、证据检索、引用问答、原文定位、项目协作、Git
审阅和可恢复任务执行。项目以离线可运行保证可评审，以 OpenAI 兼容接口提供真实模型扩展，以证据和检查点减少幻觉与误操作。源码、环境模板、依赖、技术文档和验收记录已经整理为可上传材料；完成 GitHub/Gitee 远程仓库和公网演示配置后即可进入最终提交。